

GALAGA 2D

Passo a passo na Unity

Guia para executar em casa, um passo de cada vez - com todos os scripts em C#.

SENAI Sete Lagoas - Codificação de Sistemas de Jogos Digitais - Situação de Aprendizagem Jogo_Galaga

Como usar este guia

- Faça **um passo de cada vez** e só avance quando o quadro **TESTE** do passo estiver todo funcionando.
- Os scripts estão nas caixas escuras. O nome do arquivo precisa ser **igual ao nome da classe** (ex.: GameManager.cs). Os mesmos arquivos .cs vão num .zip separado: copiar do PDF pode bagunçar a indentação.
- Fez tudo certo e deu erro? Veja a seção **Erros comuns** no final.
- Ao fim de cada passo, faça um commit (ou copie a pasta do projeto). Assim você nunca perde o que já funciona.
- Foco no **MVP do GDD**: nave, tiro, 3 inimigos, vidas, pontos, fase completa e Game Over. Boss, power-ups e sons vêm depois, e se faltar tempo são os primeiros a serem cortados.

Cronograma sugerido (final de semana)

Passo	O que fazer	Tempo	Sprint do GDD
0	Preparação do projeto	20 min	-
1	Cena, HUD e GameManager	25 min	Sprint 3
2	Nave e movimento	15 min	Sprint 1
3	Sistema de tiro	20 min	Sprint 1
4	Inimigo A e Spawner	30 min	Sprint 2
5	Inimigos B e Elite + tiro inimigo	25 min	Sprint 2
6	Telas: Game Over, Pause e Vitória	25 min	Sprint 3
7	Fases (testar e ajustar)	15 min	Sprint 3
SÁBADO	Passos 0 a 7: jogo jogável com 3 fases	~3 h	
8	Power-ups	30 min	Sprint 4
9	Boss e Vitória	30 min	Sprint 5
10	Menu principal e Build Settings	20 min	-
11	Sons e música	20 min	Sprint 4
12	Testes finais e build	30 min	Sprint 5 e 6
13	Entender o código para apresentar	30 min	-
DOMINGO	Passos 8 a 13: jogo completo, build e apresentação	~2 h40	

Os tempos são só de execução. Some uma margem para bugs: na prática, conte de 3h30 a 4h por dia.

Visão geral do projeto

Item	Conteúdo
Cenas	Menu (índice 0) e Game (índice 1).
Tags	Player (já existe), PlayerBullet e EnemyBullet (você cria).
Scripts (9)	Sfx, GameManager, PlayerController, Bullet, Enemy, EnemySpawner, PowerUp, Boss, MenuManager.
Prefabs (9)	Bullet, EnemyBullet, EnemyA, EnemyB, EnemyElite, PU_DoubleShot, PU_Shield, PU_Life, Boss.
Controles	A / seta esquerda: esquerda D / seta direita: direita Espaço: atirar Esc: pausar.
Pontuação	Inimigo A 100 B 200 Elite 500 Boss 5000 Power-up 50.
Regra de ouro	Objeto que colide = Rigidbody2D (Gravity Scale 0) + Collider 2D com Is Trigger .

Referência rápida dos prefabs

Objeto	Tag	Script e valores principais
Player	Player	PlayerController: Speed 8, Limit X 8, Fire Point, Shield Visual, Bullet Prefab
Bullet	PlayerBullet	Bullet: Direction (0, 1), Speed 12
EnemyBullet	EnemyBullet	Bullet: Direction (0, -1), Speed 6
EnemyA	-	Enemy: Type A, Health 1, Points 100, Speed 1.5, Shoot Interval 0
EnemyB	-	Enemy: Type B, Health 2, Points 200, Speed 1.5, Zigzag 3 / 2, Shoot Interval 2.5, Bullet = EnemyBullet
EnemyElite	-	Enemy: Type Elite, Health 4, Points 500, Speed 3, Shoot Interval 0
PU_DoubleShot / PU_Shield / PU_Life	-	PowerUp: Type correspondente, Fall Speed 2, Pickup Sound
Boss	-	Boss: Health 40, Speed 3, Limit X 6, Shoot Interval 1.2, Bullet = EnemyBullet

Na hora de pedir ajuda para a IA

Cole sempre: (1) a versão da Unity, (2) o script inteiro com problema e (3) a mensagem vermelha do Console. Diga também o que você esperava que acontecesse. Peça uma correção por vez e teste antes de pedir a próxima.

Objetivo: Deixar o projeto Unity pronto e organizado para receber o jogo.

1. Abra o Unity Hub > **New project** > modelo **2D** (qualquer modelo 2D serve). Nome do projeto: **Galaga2D**.
2. Na janela Project, dentro de **Assets**, crie as pastas **Scripts**, **Prefabs**, **Sprites**, **Audio** e **Scenes** (botão direito > Create > Folder).
3. Arraste suas imagens para **Sprites** e seus sons para **Audio**. Dê nomes simples: nave, inimigoA, inimigoB, inimigoElite, boss, tiro, tiroInimigo, powerDuplo, powerEscudo, powerVida, fundo. Sons: sfxTiro, sfxExplosao, sfxPowerUp, sfxDano, sfxGameOver, musicaMenu, musicaFase, musicaBoss, musicaVitoria.
4. Clique numa imagem e confira no Inspector: **Texture Type = Sprite (2D and UI)**. Se for pixel art, use Filter Mode = Point (no filter).
5. Teclado: Edit > Project Settings > Player > Other Settings > **Active Input Handling = Both** (ou Input Manager (Old)). A Unity pede para reiniciar: aceite. Nossos scripts usam `Input.GetKey`; sem isso aparece erro no Console.
6. Crie as tags: clique em qualquer objeto > no Inspector, em **Tag** escolha **Add Tag...** > botão **+** > crie **PlayerBullet** e **EnemyBullet**. (A tag **Player** já existe.)
7. Renomeie a cena **SampleScene** para **Game** (botão direito > Rename) e salve com Ctrl+S.
8. Faça backup: se usar Git, rode `git init` e faça o primeiro commit; se não, copie a pasta do projeto ao fim de cada passo.

TESTE - so avance quando tudo abaixo funcionar

- [] O projeto abre sem erros vermelhos no Console.
- [] As pastas existem e os assets estão organizados.
- [] A cena se chama Game e as tags PlayerBullet e EnemyBullet existem.

Regra de ouro das colisões neste projeto

Todo objeto que colide (nave, tiros, inimigos, power-ups, boss) precisa de **Rigidbody2D com Gravity Scale = 0** e de um **Collider 2D com Is Trigger marcado**. Quase todo bug de 'não acontece nada ao encostar' vem de esquecer um desses dois.

Objetivo: Montar a tela de pontos/vidas/fase/recorde e o script que guarda o estado do jogo.

1. Selecione a **Main Camera**: Projection = **Orthographic**, Size = **5**, Clear Flags = Solid Color, Background = preto.
2. (Opcional) Arraste a sprite **fundo** para a cena e deixe **Order in Layer** = **-10** no Sprite Renderer; ajuste a escala para cobrir a tela.
3. Hierarchy: botão direito > **UI** > **Canvas**. No Canvas Scaler: UI Scale Mode = **Scale With Screen Size**, Reference Resolution = **1920 x 1080**.
4. Dentro do Canvas crie 4 textos: botão direito no Canvas > **UI** > **Text - TextMeshPro**. Se aparecer a janela do TMP, clique em **Import TMP Essentials**. Nomes e posições: **ScoreText** ("Pontos: 0", topo esquerdo), **LevelText** ("Fase: 1", topo centro), **HighScoreText** ("Recorde: 0", topo direito) e **LivesText** ("Vidas: 3", canto inferior esquerdo). Fonte tamanho ~36, cor branca.
5. Hierarchy: botão direito > **Create Empty**, nome **GameManager**.
6. Em Assets/Scripts crie dois scripts (botão direito > Create > **MonoBehaviour Script** / C# Script) com os nomes **exatos Sfx** e **GameManager**. Abra cada um e **substitua todo o conteúdo** pelo código abaixo.

Sfx.cs

```
using UnityEngine;

// Toca um efeito sonoro uma vez (nao precisa de AudioSource no objeto)
public static class Sfx
{
    public static void Play(AudioClip clip, float volume = 1f)
    {
        if (clip == null) return;
        AudioSource.PlayClipAtPoint(clip, Camera.main.transform.position, volume);
    }
}
```

GameManager.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;

    [Header("Estado")]
    public int score = 0;
    public int lives = 3;
    public int level = 1;

    [Header("HUD")]
    public TextMeshProUGUI scoreText;
    public TextMeshProUGUI livesText;
    public TextMeshProUGUI levelText;
    public TextMeshProUGUI highScoreText;

    [Header("Telas")]
    public GameObject gameOverPanel;
    public GameObject victoryPanel;
    public GameObject pausePanel;

    [Header("Audio")]
    public AudioSource musicSource;
    public AudioClip levelMusic;
    public AudioClip bossMusic;
    public AudioClip victoryMusic;
    public AudioClip gameOverSound;

    int highScore;
    bool gameEnded;
    bool paused;
```

```

void Awake()
{
    Instance = this;
    Time.timeScale = 1f;
    highScore = PlayerPrefs.GetInt("HighScore", 0);
}

void Start()
{
    Show(gameOverPanel, false);
    Show(victoryPanel, false);
    Show(pausePanel, false);
    PlayMusic(level >= 4 ? bossMusic : levelMusic);
    UpdateHUD();
}

void Update()
{
    if (Input.GetKeyDown(KeyCode.Escape) && !gameEnded) TogglePause();
}

public void AddScore(int points)
{
    score += points;
    if (score > highScore)
    {
        highScore = score;
        PlayerPrefs.SetInt("HighScore", highScore);
    }
    UpdateHUD();
}

public void AddLife()
{
    lives++;
    UpdateHUD();
}

public void LoseLife()
{
    lives--;
    UpdateHUD();
    if (lives <= 0) GameOver();
}

public void NextLevel()
{
    level++;
    if (level >= 4) PlayMusic(bossMusic);
    UpdateHUD();
}

void GameOver()
{
    gameEnded = true;
    if (musicSource != null) musicSource.Stop();
    Sfx.Play(gameOverSound);
    Show(gameOverPanel, true);
    Time.timeScale = 0f;
}

public void Victory()
{
    gameEnded = true;
    PlayMusic(victoryMusic);
    Show(victoryPanel, true);
    Time.timeScale = 0f;
}

public void TogglePause()
{
    paused = !paused;
    Time.timeScale = paused ? 0f : 1f;
    Show(pausePanel, paused);
}

```

GameManager.cs

```
}

// Funcoes chamadas pelos botoes (On Click)
public void Restart()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}

public void GoToMenu()
{
    SceneManager.LoadScene("Menu");
}

void PlayMusic(AudioClip clip)
{
    if (musicSource == null || clip == null) return;
    musicSource.clip = clip;
    musicSource.loop = true;
    musicSource.Play();
}

void UpdateHUD()
{
    if (scoreText != null) scoreText.text = "Pontos: " + score;
    if (livesText != null) livesText.text = "Vidas: " + lives;
    if (levelText != null) levelText.text = level >= 4 ? "Fase: BOSS" : "Fase: " + level;
    if (highScoreText != null) highScoreText.text = "Recorde: " + highScore;
}

void Show(GameObject obj, bool value)
{
    if (obj != null) obj.SetActive(value);
}
}
```

7. Arraste o script **GameManager** para o objeto GameManager (Hierarchy).
8. No Inspector do GameManager, arraste os 4 textos do Canvas para **Score Text**, **Lives Text**, **Level Text** e **High Score Text**. Deixe o resto vazio por enquanto.

TESTE - so avance quando tudo abaixo funcionar

- [] Ao dar Play, o HUD mostra: Pontos: 0 / Vidas: 3 / Fase: 1 / Recorde: 0.
- [] Nenhum erro vermelho no Console.

Se o Console reclamar do TMPPro

Vá em Window > TextMeshPro > **Import TMP Essential Resources** e tente de novo.

Objetivo: Ter a nave na tela andando só na horizontal (A/D e setas).

1. Arraste a sprite **nave** para a Hierarchy e renomeie para **Player**. Position = (0, -4, 0). Ajuste a **Scale** até a nave ter cerca de 1 unidade de largura (ex.: 0.5).
2. No Inspector: **Tag = Player**. **Add Component > Rigidbody2D** (Gravity Scale = 0; em Constraints marque Freeze Rotation Z). **Add Component > Box Collider 2D** com **Is Trigger** marcado.
3. Botão direito em Player > **Create Empty**, nome **FirePoint**, Position local (0, 0.6, 0) - fica na ponta da nave (ajuste ao tamanho).
4. Botão direito em Player > **2D Object > Sprites > Circle**, nome **Shield**. Cor azul clara com Alpha ~100, escala para cobrir a nave. **Desmarque o checkbox** no topo do Inspector (objeto desativado). Se não existir Circle, use qualquer sprite.
5. Crie o script **PlayerController** e cole o código abaixo.

PlayerController.cs

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    [Header("Movimento")]
    public float speed = 8f;
    public float limitX = 8f;

    [Header("Tiro")]
    public GameObject bulletPrefab;
    public Transform firePoint;
    public float doubleShotCooldown = 0.25f;

    [Header("Extras")]
    public GameObject shieldVisual;

    [Header("Sons")]
    public AudioClip shootSound;
    public AudioClip hitSound;

    GameObject currentBullet;
    SpriteRenderer sr;
    float doubleShotUntil;
    float shieldUntil;
    float invincibleUntil;
    float nextShotTime;

    void Start()
    {
        sr = GetComponent<SpriteRenderer>();
    }

    void Update()
    {
        if (Time.timeScale == 0f) return;    // jogo pausado ou acabou

        Move();
        if (Input.GetKeyDown(KeyCode.Space)) Shoot();
        UpdateVisuals();
    }

    void Move()
    {
        float h = 0f;
        if (Input.GetKey(KeyCode.A) || Input.GetKey(KeyCode.LeftArrow)) h -= 1f;
        if (Input.GetKey(KeyCode.D) || Input.GetKey(KeyCode.RightArrow)) h += 1f;

        transform.Translate(Vector3.right * h * speed * Time.deltaTime);

        Vector3 p = transform.position;
        p.x = Mathf.Clamp(p.x, -limitX, limitX);
        transform.position = p;
    }
}
```

```

void Shoot()
{
    if (bulletPrefab == null) return;
    Vector3 pos = firePoint != null ? firePoint.position : transform.position;

    if (Time.time < doubleShotUntil)           // power-up Tiro Duplo ativo
    {
        if (Time.time < nextShotTime) return;
        nextShotTime = Time.time + doubleShotCooldown;
        Instantiate(bulletPrefab, pos + Vector3.left * 0.3f, Quaternion.identity);
        Instantiate(bulletPrefab, pos + Vector3.right * 0.3f, Quaternion.identity);
    }
    else                                         // normal: so 1 tiro por vez
    {
        if (currentBullet != null) return;
        currentBullet = Instantiate(bulletPrefab, pos, Quaternion.identity);
    }
    Sfx.Play(shootSound);
}

public void ActivateDoubleShot(float seconds)
{
    doubleShotUntil = Time.time + seconds;
}

public void ActivateShield(float seconds)
{
    shieldUntil = Time.time + seconds;
}

public void TakeDamage()
{
    if (Time.time < invincibleUntil) return;

    if (Time.time < shieldUntil)               // escudo absorve 1 ataque
    {
        shieldUntil = 0f;
        invincibleUntil = Time.time + 0.5f;
        return;
    }

    Sfx.Play(hitSound);
    invincibleUntil = Time.time + 2f;           // 2s piscando, sem levar dano
    GameManager.Instance.LoseLife();
}

void UpdateVisuals()
{
    if (shieldVisual != null) shieldVisual.SetActive(Time.time < shieldUntil);

    if (sr != null)
    {
        bool blinkOff = Time.time < invincibleUntil
                        && Mathf.FloorToInt(Time.time * 10f) % 2 == 1;
        sr.enabled = !blinkOff;
    }
}
}

```

6. Arraste o script para o objeto **Player**. No Inspector: **Fire Point** = FirePoint, **Shield Visual** = Shield, **Shoot Sound** = sfxTiro, **Hit Sound** = sfxDano. Deixe **Bullet Prefab** vazio por enquanto.

TESTE - so avance quando tudo abaixo funcionar

- [] A e seta esquerda movem para a esquerda; D e seta direita movem para a direita.
- [] A nave para nas bordas da tela (se sair ou parar cedo, ajuste **Limit X**).

Objetivo: Fazer a nave atirar: 1 tiro por vez, que some ao sair da tela.

1. Arraste a sprite **tiro** para a Hierarchy, renomeie para **Bullet** e ajuste a Scale (~0.4).
2. No Inspector: **Tag = PlayerBullet**; **Rigidbody2D** (Gravity Scale = 0); **Box Collider 2D** com **Is Trigger** marcado.
3. Crie o script **Bullet** e cole o código abaixo.

Bullet.cs

```
using UnityEngine;

// Serve para o tiro do jogador (direction 0,1) e dos inimigos (direction 0,-1)
public class Bullet : MonoBehaviour
{
    public Vector2 direction = Vector2.up;
    public float speed = 12f;

    void Update()
    {
        transform.Translate((Vector3)direction * speed * Time.deltaTime);

        Vector3 p = transform.position;
        if (Mathf.Abs(p.x) > 12f || Mathf.Abs(p.y) > 7f) Destroy(gameObject);
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        // Tiro inimigo atingindo o jogador
        if (CompareTag("EnemyBullet") && other.CompareTag("Player"))
        {
            PlayerController player = other.GetComponent<PlayerController>();
            if (player != null) player.TakeDamage();
            Destroy(gameObject);
        }
    }
}
```

4. Arraste o script para o objeto Bullet. Valores: **Direction = (0, 1)** e **Speed = 12**.
5. Arraste o Bullet da Hierarchy para a pasta **Prefabs** (vira prefab) e **apague** o Bullet da cena.
6. Selecione o Player e arraste o prefab Bullet para o campo **Bullet Prefab** do PlayerController.

TESTE - so avance quando tudo abaixo funcionar

- [] Espaço dispara um tiro que sobe.
- [] Só dá para atirar de novo quando o tiro anterior sair da tela.
- [] Os tiros somem fora da tela (a Hierarchy não acumula Bullet(Clone)).

Objetivo: Criar o primeiro inimigo e o gerador de ondas. Ao final a Fase 1 já é jogável.

1. Arraste a sprite **inimigoA** para a Hierarchy, renomeie para **EnemyA**, Scale ~0.5. Adicione **Rigidbody2D** (Gravity Scale = 0) e **Box Collider 2D** com **Is Trigger**.
2. Crie os scripts **Enemy** e **EnemySpawner** e cole os códigos abaixo.

Enemy.cs

```
using UnityEngine;

public enum EnemyType { A, B, Elite }

public class Enemy : MonoBehaviour
{
    public EnemyType type = EnemyType.A;
    public int health = 1;
    public int points = 100;
    public float speed = 1.5f;

    [Header("Zigue-zague (tipo B)")]
    public float zigzagSpeed = 3f;
    public float zigzagWidth = 2f;

    [Header("Tiro (0 = nao atira)")]
    public float shootInterval = 0f;
    public GameObject bulletPrefab;

    [Header("Som")]
    public AudioClip explosionSound;

    float startX;
    float birthTime;
    float nextShootTime;
    bool dead;

    void Start()
    {
        startX = transform.position.x;
        birthTime = Time.time;
        nextShootTime = Time.time + Random.Range(1f, 2.5f);
    }

    void Update()
    {
        Vector3 p = transform.position;
        p.y -= speed * Time.deltaTime;

        if (type == EnemyType.B)
            p.x = startX + Mathf.Sin((Time.time - birthTime) * zigzagSpeed) * zigzagWidth;

        transform.position = p;

        if (shootInterval > 0f && bulletPrefab != null && Time.time >= nextShootTime)
        {
            Instantiate(bulletPrefab, transform.position, Quaternion.identity);
            nextShootTime = Time.time + shootInterval;
        }

        if (p.y < -6f) Leave(); // passou da nave e saiu da tela
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("PlayerBullet"))
        {
            Destroy(other.gameObject);
            TakeDamage(1);
        }
        else if (other.CompareTag("Player")) // bateu na nave
        {
            PlayerController player = other.GetComponent<PlayerController>();
```

Enemy.cs

```
        if (player != null) player.TakeDamage();
        Die(false);
    }

    public void TakeDamage(int amount)
    {
        health -= amount;
        if (health <= 0) Die(true);
    }

    void Die(bool byPlayer)
    {
        if (dead) return;
        dead = true;

        if (byPlayer)
        {
            GameManager.Instance.AddScore(points);
            EnemySpawner.Instance.TryDropPowerUp(transform.position);
        }
        Sfx.Play(explosionSound);
        Finish();
    }

    void Leave()
    {
        if (dead) return;
        dead = true;
        Finish();
    }

    void Finish()
    {
        EnemySpawner.Instance.EnemyGone();
        Destroy(gameObject);
    }
}
```

EnemySpawner.cs

```
using UnityEngine;
using System.Collections;

public class EnemySpawner : MonoBehaviour
{
    public static EnemySpawner Instance;

    [Header("Prefabs")]
    public GameObject enemyA;
    public GameObject enemyB;
    public GameObject enemyElite;
    public GameObject bossPrefab;
    public GameObject[] powerUps;

    [Header("Area de spawn")]
    public float spawnY = 6f;
    public float limitX = 6f;

    int alive;        // inimigos vivos na tela

    void Awake()
    {
        Instance = this;
    }

    void Start()
    {
        StartCoroutine(LevelRoutine(GameManager.Instance.level));
    }

    IEnumerator LevelRoutine(int level)
    {
        yield return new WaitForSeconds(2f);
    }
}
```

```

    if (level >= 4) // fase do Boss
    {
        Instantiate(bossPrefab, new Vector3(0f, 3.5f, 0f), Quaternion.identity);
        yield break;
    }

    int count = 8 + level * 4; // fase 1: 12, fase 2: 16, fase 3: 20
    float interval = 1.4f - level * 0.2f; // fase 1: 1.2s, fase 2: 1.0s, fase 3: 0.8s

    for (int i = 0; i < count; i++)
    {
        SpawnEnemy(ChooseEnemy(level), level);
        yield return new WaitForSeconds(interval);
    }

    yield return new WaitUntil(() => alive <= 0); // espera limpar a tela

    GameManager.Instance.NextLevel();
    StartCoroutine(LevelRoutine(GameManager.Instance.level));
}

GameObject ChooseEnemy(int level)
{
    float r = Random.value;
    if (level == 1) return enemyA;
    if (level == 2) return r < 0.6f ? enemyB : enemyB;
    return r < 0.4f ? enemyA : (r < 0.75f ? enemyB : enemyElite);
}

void SpawnEnemy(GameObject prefab, int level)
{
    Vector3 pos = new Vector3(Random.Range(-limitX, limitX), spawnY, 0f);
    GameObject go = Instantiate(prefab, pos, Quaternion.identity);
    go.GetComponent<Enemy>().speed *= 1f + (level - 1) * 0.35f; // fica mais rapido
    alive++;
}

public void EnemyGone()
{
    alive--;
}

public void TryDropPowerUp(Vector3 pos)
{
    if (powerUps == null || powerUps.Length == 0) return;

    float chance = GameManager.Instance.level >= 3 ? 0.25f : 0.12f;
    if (Random.value < chance)
        Instantiate(powerUps[Random.Range(0, powerUps.Length)], pos, Quaternion.identity);
}
}

```

3. Arraste o script **Enemy** para o EnemyA. Valores: **Type = A, Health = 1, Points = 100, Speed = 1.5, Shoot Interval = 0, Explosion Sound = sfxExplosao**.
4. Arraste o EnemyA para a pasta **Prefabs** e apague-o da cena.
5. Hierarchy > Create Empty, nome **EnemySpawner**, posição (0, 0, 0). Arraste o script EnemySpawner para ele. Preencha **Enemy A** com o prefab EnemyA; Spawn Y = 6; Limit X = 6. Deixe o resto vazio por enquanto.

TESTE - so avance quando tudo abaixo funcionar

- [] Depois de ~2 s aparecem inimigos no topo, em posições aleatórias, descendo.
- [] Seu tiro destrói o inimigo: Pontos +100 e o Recorde sobe junto.
- [] Se um inimigo encosta na nave, Vidas -1 e a nave pisca por 2 s.
- [] Quando todos os 12 inimigos acabam, a fase passa para 2 (o HUD muda).

Esperado neste ponto

Ao chegar na Fase 2 vai aparecer um erro no Console por causa do Enemy B vazio. É normal: ele é criado no próximo passo. Se as vidas chegarem a 0 o jogo congela; a tela de Game Over vem no Passo 6.

Objetivo: Criar os outros dois tipos de inimigo e o tiro dos inimigos.

1. **EnemyBullet:** arraste a sprite **tirolnimigo** para a Hierarchy, nome **EnemyBullet**, Scale ~0.4. **Tag = EnemyBullet**, Rigidbody2D (Gravity 0), Box Collider 2D com **Is Trigger**. Adicione o script **Bullet** com **Direction = (0, -1)** e **Speed = 6**. Vire prefab e apague da cena.
2. **EnemyB:** na pasta Prefabs, selecione EnemyA e aperte **Ctrl+D** (duplicar); renomeie para **EnemyB**. No Sprite Renderer troque a sprite por **inimigoB**. No script Enemy: **Type = B**, **Health = 2**, **Points = 200**, **Speed = 1.5**, **Zigzag Speed = 3**, **Zigzag Width = 2**, **Shoot Interval = 2.5**, **Bullet Prefab = EnemyBullet**.
3. **EnemyElite:** duplique de novo, nome **EnemyElite**, sprite **inimigoElite**. Enemy: **Type = Elite**, **Health = 4**, **Points = 500**, **Speed = 3**, **Shoot Interval = 0**.
4. Selecione o objeto EnemySpawner na cena e preencha **Enemy B** e **Enemy Elite** com os prefabs.

TESTE - so avance quando tudo abaixo funcionar

- [] Selecione o GameManager e mude **Level = 2** no Inspector, depois dê Play: aparecem A e B; o B balança em zigue-zague e atira.
- [] O tiro inimigo que acerta a nave tira 1 vida.
- [] Com **Level = 3** o Elite aparece, é rápido e leva 4 tiros.
- [] Volte o Level do GameManager para **1** antes de continuar.

Não esqueça!

Level no Inspector serve só para testar. Se esquecer em 2 ou 3, o jogo começa na fase errada.

Objetivo: Criar os painéis e ligar os botões ao GameManager.

1. No Canvas: botão direito > **UI > Panel**, nome **GameOverPanel**. Deixe a cor escura (Alpha ~200). Dentro dele crie um texto **"GAME OVER"** grande e dois botões: **UI > Button - TextMeshPro** com os textos "Jogar de novo" e "Menu".
2. Duplique o painel (Ctrl+D) duas vezes: **PausePanel** (texto "PAUSADO"; botões "Continuar" e "Menu") e **VictoryPanel** (texto "VITORIA!"; botões "Jogar de novo" e "Menu").
3. **Ligando os botões:** selecione o botão > no Inspector, em **On Click ()** clique em + > arraste o objeto **GameManager** (da Hierarchy, não o script) para o campo > escolha a função: "Jogar de novo" = **GameManager > Restart**; "Menu" = **GameManager > GoToMenu**; "Continuar" = **GameManager > TogglePause**.
4. Selecione o GameManager e arraste os painéis para **Game Over Panel, Victory Panel, Pause Panel**. Em **Game Over Sound** coloque sfxGameOver.
5. (Opcional) Desative os 3 painéis (checkbox no topo do Inspector) para editar a cena sem eles cobrindo a tela. O GameManager os esconde sozinho no Play.

TESTE - so avance quando tudo abaixo funcionar

- [] ESC pausa o jogo e mostra PAUSADO; "Continuar" (ou ESC de novo) volta a jogar.
- [] Ao perder as 3 vidas, aparece GAME OVER, o jogo congela e toca o som.
- [] "Jogar de novo" reinicia com 3 vidas e 0 pontos (o recorde permanece).
- [] "Menu" ainda dá erro no Console (a cena Menu só existe no Passo 10): normal.

Objetivo: A lógica das fases já está no EnemySpawner. Aqui você testa a progressão inteira e ajusta a dificuldade.

1. Com **Level = 1**, dê Play e jogue de verdade: mate todos os inimigos e confirme que o HUD vai para **Fase: 2**, depois **Fase: 3**.
2. Ajuste a dificuldade se achar fácil ou difícil mexendo no **EnemySpawner.cs**: `count` (quantidade), `interval` (tempo entre inimigos) e o `0.35f` em `SpawnEnemy` (quanto a velocidade aumenta por fase).

TESTE - so avance quando tudo abaixo funcionar

- [] Fase 1: só inimigo A, aprendizado dos controles.
- [] Fase 2: mais rápidos e com tiros (inimigo B).
- [] Fase 3: Elite aparece e a fila de ataque é mais intensa.
- [] A passagem de fase só acontece quando a tela está limpa.

Resumo das fases

Fase 1: 12 inimigos A | Fase 2: 16 (A e B) | Fase 3: 20 (A, B e Elite, power-ups mais frequentes) | Fase 4: Boss.

Objetivo: Tiro Duplo (15 s), Escudo (10 s, absorve 1 ataque) e Vida Extra (+1 vida).

1. Crie o script **PowerUp** e cole o código abaixo.

PowerUp.cs

```
using UnityEngine;

public enum PowerUpType { DoubleShot, Shield, ExtraLife }

public class PowerUp : MonoBehaviour
{
    public PowerUpType type;
    public float fallSpeed = 2f;
    public AudioClip pickupSound;

    void Update()
    {
        transform.Translate(Vector3.down * fallSpeed * Time.deltaTime);
        if (transform.position.y < -6f) Destroy(gameObject);
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.CompareTag("Player")) return;

        PlayerController player = other.GetComponent<PlayerController>();
        if (player == null) return;

        switch (type)
        {
            case PowerUpType.DoubleShot: player.ActivateDoubleShot(15f); break;
            case PowerUpType.Shield:     player.ActivateShield(10f);   break;
            case PowerUpType.ExtraLife:  GameManager.Instance.AddLife(); break;
        }

        GameManager.Instance.AddScore(50);
        Sfx.Play(pickupSound);
        Destroy(gameObject);
    }
}
```

2. Crie 3 objetos a partir das sprites **powerDuplo**, **powerEscudo**, **powerVida** (se tiver só uma sprite, use a mesma e mude a **Color**). Nomes: **PU_DoubleShot**, **PU_Shield**, **PU_Life**. Scale ~0.4.
3. Em cada um: **Rigidbody2D** (Gravity 0), **Box Collider 2D** com **Is Trigger** e o script **PowerUp**. Em **Type** escolha DoubleShot / Shield / ExtraLife; Fall Speed = 2; Pickup Sound = sfxPowerUp.
4. Arraste os 3 para a pasta Prefabs e apague da cena.
5. No EnemySpawner, em **Power Ups**: coloque **Size = 3** e arraste os 3 prefabs.

TESTE - so avance quando tudo abaixo funcionar

- [] Coletar o Tiro Duplo: 2 projéteis por disparo durante 15 s, depois volta ao normal.
- [] Escudo: aparece a bolha azul; absorve 1 ataque e some (ou some após 10 s).
- [] Vida Extra: Vidas +1.
- [] Cada power-up coletado dá +50 pontos.

Truque para testar rápido

Em EnemySpawner.cs, na função TryDropPowerUp, troque 0.25f e 0.12f por 1f (todo inimigo morto solta power-up). Depois de testar, **devolva os valores originais**.

Objetivo: Criar o chefe da Fase 4: muita vida, tiros em leque, 5000 pontos e a tela de vitória.

1. Crie o script **Boss** e cole o código abaixo.

Boss.cs

```
using UnityEngine;

public class Boss : MonoBehaviour
{
    public int health = 40;
    public float speed = 3f;
    public float limitX = 6f;
    public float shootInterval = 1.2f;
    public GameObject bulletPrefab;
    public AudioClip explosionSound;

    int direction = 1;
    float nextShootTime;
    bool dead;

    void Update()
    {
        transform.Translate(Vector3.right * direction * speed * Time.deltaTime);
        if (transform.position.x > limitX) direction = -1;
        if (transform.position.x < -limitX) direction = 1;

        if (Time.time >= nextShootTime)
        {
            nextShootTime = Time.time + shootInterval;
            Fire();
        }
    }

    void Fire() // leque de 5 tiros
    {
        for (int i = -2; i <= 2; i++)
        {
            GameObject b = Instantiate(bulletPrefab, transform.position, Quaternion.identity);
            b.GetComponent<Bullet>().direction = new Vector2(i * 0.3f, -1f).normalized;
        }
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (dead || !other.CompareTag("PlayerBullet")) return;

        Destroy(other.gameObject);
        health--;
        if (health > 0) return;

        dead = true;
        GameManager.Instance.AddScore(5000);
        Sfx.Play(explosionSound);
        Destroy(gameObject);
        GameManager.Instance.Victory();
    }
}
```

2. Arraste a sprite **boss** para a Hierarchy, nome **Boss**, Scale ~1.5. **Rigidbody2D** (Gravity 0) e **Box Collider 2D** com **Is Trigger**.
3. Adicione o script Boss: **Health = 40, Speed = 3, Limit X = 6, Shoot Interval = 1.2, Bullet Prefab = EnemyBullet, Explosion Sound = sfxExplosao**. Vire prefab e apague da cena.
4. No EnemySpawner, preencha **Boss Prefab** com o prefab Boss.
5. Para testar: GameManager > **Level = 4** e Play.

TESTE - so avance quando tudo abaixo funcionar

- [] O Boss aparece após ~2 s, vai de um lado a outro e atira 5 tiros em leque.
- [] Cada acerto tira 1 de vida do boss.
- [] Ao derrotá-lo: +5000 pontos, painel de VITORIA e música de vitória.
- [] Volte o **Level para 1** e jogue o fluxo completo: Fase 1 > 2 > 3 > Boss > Vitória.

Boss muito fácil ou difícil?

Ajuste **Health** (vida), **Shoot Interval** (frequência dos tiros) e **Speed** no Inspector do prefab Boss. Não precisa mexer no código.

Objetivo: Criar a cena Menu (Jogar / Sair) e registrar as duas cenas para o build.

1. File > **New Scene** (modelo 2D/Basic) e salve como **Menu** em Assets/Scenes. Câmera preta (ou use a sprite de fundo).
2. Crie um Canvas (Scale With Screen Size, 1920 x 1080) com: título "**GALAGA 2D**" (TextMeshPro grande), um texto **HighScoreText** ("Recorde: 0") e dois botões: **Jogar** e **Sair**.
3. Crie o script **MenuManager** e cole o código abaixo.

MenuManager.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;

public class MenuManager : MonoBehaviour
{
    public TextMeshProUGUI highScoreText;

    void Start()
    {
        Time.timeScale = 1f;
        if (highScoreText != null)
            highScoreText.text = "Recorde: " + PlayerPrefs.GetInt("HighScore", 0);
    }

    public void Play()
    {
        SceneManager.LoadScene("Game");
    }

    public void Quit()
    {
        Application.Quit(); // so funciona no jogo compilado (build)
    }
}
```

4. Hierarchy > Create Empty, nome **MenuManager**, e arraste o script. Arraste o HighScoreText para o campo do script.
5. Botão **Jogar**: On Click > + > objeto MenuManager > **MenuManager > Play**. Botão **Sair**: **MenuManager > Quit**.
6. Abra File > **Build Profiles** (Unity 6) ou **Build Settings** (versões anteriores). Com a cena Menu aberta clique **Add Open Scenes**; abra a cena Game e clique de novo. Ordem final: **0 = Menu, 1 = Game**.

TESTE - so avance quando tudo abaixo funcionar

- [] Abrindo a cena Menu e dando Play, o botão Jogar leva ao jogo.
- [] No Game Over (ou na Vitória) o botão Menu volta ao menu.
- [] O recorde aparece no menu e continua salvo após reiniciar o Play.

Dica

O botão Sair só funciona no jogo compilado (build); no editor não faz nada. Se os botões não clicarem, confira se existe o objeto **EventSystem** na Hierarchy.

Objetivo: Ligar música de menu, fase, boss e vitória, e conferir todos os efeitos.

1. Cena **Game**: Create Empty, nome **Musica**, Add Component > **Audio Source**: **Play On Awake** desmarcado e **Loop** marcado.
2. Selecione o GameManager: **Music Source = Musica**, **Level Music = musicaFase**, **Boss Music = musicaBoss**, **Victory Music = musicaVitoria**.
3. Cena **Menu**: Create Empty, nome **MusicaMenu**, Audio Source com **AudioClip = musicaMenu**, **Play On Awake** e **Loop** marcados.
4. Confira se todos os campos de som estão preenchidos: Player (Shoot Sound, Hit Sound), os 3 prefabs de inimigo e o Boss (Explosion Sound), os 3 power-ups (Pickup Sound) e o GameManager (Game Over Sound).

TESTE - so avance quando tudo abaixo funcionar

[] Ouço o tiro, a explosão, a coleta de power-up, o dano e o Game Over.

[] A música muda: menu, fase, boss e vitória.

Objetivo: Passar o Plano de Testes do GDD, corrigir bugs e gerar o jogo para Windows.

1. Jogue do Menu até a Vitória e marque a tabela do Plano de Testes (próxima seção).
2. Confira o desempenho: na aba Game, ative **Stats** e veja o FPS (meta: 60 ou mais).
3. File > **Build Profiles / Build Settings** > plataforma **Windows** > **Build** > crie uma pasta **Build**.
4. Execute o **.exe** gerado e jogue do menu até o final. Compacte a pasta Build em **.zip** para entregar.

TESTE - so avance quando tudo abaixo funcionar

- [] O jogo compilado abre em menos de 5 segundos.
- [] Todos os itens do Plano de Testes estão marcados.

Objetivo: Se o instrutor perguntar, você precisa saber explicar o que cada script faz.

1. Leia a tabela abaixo e depois abra cada script, procurando as linhas citadas.
2. Treine responder as perguntas prováveis em voz alta (2 minutos por script).

Script	O que faz	Pergunta provável e resposta
GameManager	Guarda pontos, vidas, fase e recorde; controla pausa, Game Over e Vitória.	<i>Como o recorde é salvo?</i> Com <code>PlayerPrefs.SetInt</code> , que guarda o número no computador.
PlayerController	Movimenta a nave, atira (1 por vez, ou 2 com power-up) e recebe dano.	<i>Como limita a 1 tiro?</i> Guarda o tiro em <code>currentBullet</code> ; se ele ainda existe (<code>!= null</code>), não atira.
Bullet	Movimenta o projétil e o destrói fora da tela. Tiro inimigo machuca o jogador.	<i>Por que um script serve para os dois tiros?</i> A direção e a tag mudam no Inspector.
Enemy	Desce, balança (tipo B), atira, tem vida e pontos.	<i>Como é o zigue-zague?</i> <code>Mathf.Sin</code> do tempo desloca o X para os lados.
EnemySpawner	Cria as ondas, conta inimigos vivos, passa de fase e solta power-ups.	<i>Como sabe que a fase acabou?</i> <code>WaitUntil(alive <= 0)</code> : espera o contador zerar.
PowerUp	Cai e aplica o efeito ao encostar na nave.	<i>O que é o switch?</i> Escolhe o efeito pelo Type.
Boss	Movimenta de lado a lado, atira em leque e dispara a Vitória ao morrer.	<i>Como faz o leque?</i> Um for cria 5 tiros com direções X diferentes.
MenuManager / Sfx	Botões Jogar/Sair; toca efeitos sonoros pontuais.	<i>Para que serve Sfx?</i> Evita ter um <code>AudioSource</code> em cada objeto.

Conceitos que costumam ser perguntados

- **Rigidbody2D + Is Trigger:** faz o Unity avisar (`OnTriggerEnter2D`) quando dois objetos se encostam, sem empurrar um ao outro.
- **Time.timeScale = 0:** congela o jogo (usado na pausa, Game Over e Vitória). Com 1 volta ao normal.
- **Prefab:** molde de um objeto; `Instantiate` cria cópias dele em tempo de jogo.
- **Coroutine (IEnumerator):** código que pode esperar (`yield return`) entre uma ação e outra, usada nas ondas de inimigos.
- **Singleton (Instance):** `GameManager.Instance` permite que qualquer script fale com o GameManager.

Plano de Testes do GDD (Passo 12)

Caso de teste	Resultado esperado	OK
Nave se move para a esquerda	Funcionando	[]
Nave se move para a direita	Funcionando	[]
Nave dispara tiros	Funcionando	[]
Inimigo recebe dano	Funcionando	[]
Pontuação aumenta	Funcionando	[]
Vida reduz após colisão	Funcionando	[]
Game Over aparece	Funcionando	[]
Extra: Esc pausa e continua	Funcionando	[]
Extra: Power-ups (duplo, escudo, vida)	Funcionando	[]
Extra: Fases 1, 2 e 3 progridem	Funcionando	[]
Extra: Boss aparece, atira e é derrotado	Funcionando	[]
Extra: Vitória aparece e Menu volta ao início	Funcionando	[]
Extra: Recorde fica salvo	Funcionando	[]

Requisitos funcionais x onde são atendidos

Requisito	Descrição	Passo
RF01	Mover a nave para esquerda e direita	2
RF02	Disparar projéteis	3
RF03	Inimigos se movem automaticamente	4 e 5
RF04	Sistema de colisão	3, 4 e 5
RF05	Contabilizar pontos	1 e 4
RF06	Controlar vidas	1, 2 e 5
RF07	Tela de Game Over	6
RF08	Sistema de fases	4 e 7
RNF01 a RNF04	60 FPS, carregamento menor que 5 s, interface intuitiva, Windows 10 e 11	12

Erros comuns e soluções

Sintoma	Causa provável e solução
Erro vermelho NullReferenceException	Um campo do Inspector ficou vazio (ex.: Bullet Prefab, Score Text, Boss Prefab). Clique duas vezes no erro: ele mostra a linha.
Script não pode ser anexado ao objeto	O nome do arquivo é diferente do nome da classe, ou há erro de compilação em outro script. Corrija o Console primeiro.
Erro de Input (InvalidOperationException)	Edit > Project Settings > Player > Active Input Handling = Both (Passo 0).
Tiro atravessa o inimigo / nada acontece ao encostar	Falta Rigidbody2D, falta Is Trigger no collider, ou a tag do tiro não é PlayerBullet / EnemyBullet.
Tag 'PlayerBullet' is not defined	Crie a tag em Tag > Add Tag... (Passo 0).
A nave não atira	Bullet Prefab vazio no PlayerController, ou o tiro anterior ainda está na tela (só 1 por vez).
Botão não faz nada	No On Click, arraste o objeto GameManager da Hierarchy (não o arquivo do script) e escolha a função.

Sintoma	Causa provável e solução
Texto TMP aparece quadrado ou os botões não clicam	Window > TextMeshPro > Import TMP Essential Resources; confira se existe o EventSystem.
Jogo começa na fase errada	O Level do GameManager ficou diferente de 1 após os testes.
Todos os inimigos soltam power-up	Você esqueceu de devolver os valores de chance (0.12 e 0.25) no EnemySpawner.